

# Discovery Executive Summary

Project: discovery-16july · Generated: 16/07/2026, 11:26:17

EXECUTIVE SUMMARY

This report consolidates the overall ratings, key findings, and recommended actions from the 2 discovery analyses run across this codebase (frontend and backend). Each section below reproduces that analysis's executive view; full evidence and diagrams live in the individual reports.

## Portfolio Overview

| # | Analysis                           | Overall Rating | Hotspot Score       |
|---|------------------------------------|----------------|---------------------|
| 1 | Architecture & Design Analysis     | HIGH RISK      | —                   |
| 2 | Code Quality & Complexity Analysis | HIGH RISK      | 46 / 100 — Moderate |

## 1. Architecture & Design Analysis

OVERALL CODEBASE RATING — ARCHITECTURE & DESIGN

High Risk

Driven by High-Risk domain boundary violations (H8), shared-database coupling (H9), duplicated cross-layer logic (H10), and parallel Laravel/dev-api stacks (H11).

EXECUTIVE SUMMARY

FSDKC (Klearcom voice observability platform) spans three runtimes: a Laravel 11 REST API (`backend/`, 26 PHP files), a React 18 SPA (`frontend/src/`, 15 TS/JS files), and a parallel Express dev-api (`dev-api/`, 9 JS files). Controllers are thin by LOC (avg 65), but business workflows are split inconsistently — reachability math and IVR tree building are copy-pasted across controllers, services, and dev-api. There are zero repository classes; all persistence goes through Eloquent models directly from controllers. The dominant risks are **cross-domain coupling** (Dashboard and LegacyReport controllers query both Connect and Discovery models in one schema), **shared-database coupling** (80% of business tables accessed across domains), and **parallel-stack duplication** (Laravel + dev-api mirror the same realtime/test logic). Frontend architecture is healthier (shared `api/client.ts`, Zustand, React Query hooks) but pages still embed query/mutation wiring inline and one legacy class component remains.

## 1.1 Benchmark Ratings Summary

| #  | Hotspot                                  | Primary KPI                        | GOOD | MODERATE | HIGH RISK | Measured                     | Rating    |
|----|------------------------------------------|------------------------------------|------|----------|-----------|------------------------------|-----------|
| H1 | Fat Controllers                          | Avg LOC per controller             | <150 | 150–300  | >300      | 65 LOC (6 API controllers)   | GOOD      |
| H2 | Missing Service Layer                    | Controllers accessing repos/models | <10  | 10–20    | >20       | 4 controllers                | GOOD      |
| H3 | Missing Repository Pattern               | Direct DB access points            | <10  | 10–20    | >20       | 6 files                      | GOOD      |
| H4 | Circular Dependencies                    | Dependency cycles                  | 0    | 1–3      | >3        | 2 Eloquent relation pairs    | MODERATE  |
| H5 | Shared Utility Abuse                     | Utility files w/ business logic    | 0    | 1–5      | >5        | 1 file ( LegacyDataMapper )  | MODERATE  |
| H6 | Direct SQL in Controllers                | ORM compliance %                   | >90% | 60–90%   | <60%      | 100% Eloquent                | GOOD      |
| H7 | God Classes                              | Classes >1000 LOC                  | 0    | 1–3      | >3        | 0                            | GOOD      |
| H8 | Domain Boundary Violations               | Cross-domain access points         | 0    | 1–5      | >5        | 8                            | HIGH RISK |
| H9 | Shared Database Coupling                 | Tables shared across domains       | <10% | 10–30%   | >30%      | 80% (4/5 tables)             | HIGH RISK |
| F1 | Business Logic in Components             | Avg LOC per component              | <150 | 150–300  | >300      | 75 LOC (11 components/pages) | GOOD      |
| F2 | Missing Frontend Service/Data Layer      | Components w/ inline API calls     | <10  | 10–20    | >20       | 5 pages/widgets              | GOOD      |
| F3 | God / Oversized Components               | Components >400 LOC                | 0    | 1–3      | >3        | 0                            | GOOD      |
| F4 | Prop Drilling / Global State Abuse       | Max prop-drilling depth            | ≤2   | 3–4      | >4        | 2 levels                     | GOOD      |
| F5 | Legacy / Inconsistent Component Patterns | Legacy-pattern components          | 0    | 1–10     | >10       | 1 class + 1 .jsx             | MODERATE  |

|     |                                      |                                    |   |     |    |                       |           |
|-----|--------------------------------------|------------------------------------|---|-----|----|-----------------------|-----------|
| H10 | Duplicated Domain Logic (additional) | Copy-pasted workflow sites         | 0 | 1–3 | >3 | 6 sites               | HIGH RISK |
| H11 | Parallel Runtime Stacks (additional) | Full duplicate API implementations | 0 | 1   | >1 | 2 (Laravel + dev-api) | HIGH RISK |

## 1.4 Actions Required

| Hotspot | Action                                                                                                                         | Rating    | Priority |
|---------|--------------------------------------------------------------------------------------------------------------------------------|-----------|----------|
| H4      | Encapsulate bidirectional Eloquent relations inside context-specific repositories; expose DTOs at boundaries                   | MODERATE  | MEDIUM   |
| H5      | Replace <code>extract()</code> in <code>LegacyDataMapper</code> and <code>LegacyReportController</code> with typed DTO mappers | MODERATE  | MEDIUM   |
| H8      | Split cross-domain controllers/services; introduce ACL between Connect and Discovery contexts                                  | HIGH RISK | CRITICAL |
| H9      | Assign table ownership per domain; replace cross-domain Dashboard queries with read-model projections                          | HIGH RISK | HIGH     |
| F5      | Migrate <code>LegacyMonitorPoller</code> to function component; add Error Boundary in <code>App.tsx</code>                     | MODERATE  | MEDIUM   |
| H10     | Extract <code>ReachabilityCalculator</code> and <code>IvrTreeBuilder</code> ; delete 6 duplicate implementations               | HIGH RISK | CRITICAL |
| H11     | Deprecate or proxy <code>dev-api/</code> ; designate Laravel as single API source of truth                                     | HIGH RISK | HIGH     |

## 1.5 Expected Outcomes

- Consolidating reachability and tree-building logic into domain services eliminates KPI drift between Connect checks, legacy reports, and the dev-api simulator.
- Repository interfaces and application services make controllers thin HTTP adapters testable with mocked persistence, raising confidence for schema and MongoDB changes.
- Bounded contexts with an anti-corruption layer let Connect and Discovery evolve independently — including future extraction to separate deployables.
- Retiring the parallel dev-api stack removes dual-maintenance burden and ensures local development matches production Laravel behavior.
- Frontend domain service modules and migrated legacy components produce consistent React patterns with proper cleanup and error isolation.

Full report saved to `docs/discovery/01-architecture-design.md` (orchestration UI will convert to PDF).

Pipeline artifact copy: `agent-runs/20260716T110541_bql9tn/01-architecture-design.md`.

Analysis covered **backend** (26 PHP files), **frontend** (15 TS/JS files), and **dev-api** (9 JS files) from **shende-shweta/FSDKC** via GitHub REST API.

## 2. Code Quality & Complexity Analysis

OVERALL CODEBASE RATING — CODE QUALITY & COMPLEXITY

### High Risk

Driven by H1 cyclomatic complexity (36 in ConnectPage), H3 oversized ConnectPage component (201 LOC), and H4 cross-runtime business-logic duplication (~11%).

EXECUTIVE SUMMARY

Analysis covered **50 source files** across three layers: **17 frontend** (898 LOC), **26 backend** (918 LOC), and **6 dev-api** (720 LOC), totaling **2,598 LOC** of application code. No stack-specific complexity linter (ESLint `complexity`, PHPMD, Sonar) is configured; metrics were derived via manual branch counting and LOC analysis against GitHub `main`. The codebase is young (**3 commits**) with low churn and clear single-author ownership, but **structural complexity and duplication are elevated**: `ConnectPage.tsx` registers **cyclomatic complexity ~36** and **201 LOC** in a single component, and **parallel Laravel + Express implementations** duplicate realtime test workflows and KPI logic (~11% estimated business-rule duplication). Overall health is **High Risk**, driven by frontend page complexity and cross-runtime business-logic duplication despite favorable churn and ownership signals.

### 2.1 Benchmark Ratings Summary

| #  | Hotspot                    | Primary KPI                 | GOOD | MODERATE | HIGH RISK | Measured                                      | Rating   |
|----|----------------------------|-----------------------------|------|----------|-----------|-----------------------------------------------|----------|
| H1 | High Cyclomatic Complexity | Max complexity per method   | <10  | 10–20    | >20       | 36 ( <code>ConnectPage.tsx</code> component)  | HIGH     |
| H2 | Large Classes              | Largest class LOC           | <300 | 300–1000 | >1000     | 224 ( <code>dev-api/src/server.js</code> )    | GOOD     |
| H3 | Large Functions            | Largest function LOC        | <50  | 50–200   | >200      | 201 ( <code>ConnectPage.tsx</code> component) | HIGH     |
| H4 | Business Logic Duplication | Duplicated business logic % | <5%  | 5–10%    | >10%      | ~11% (est. ~285 LOC duplicated / 2,598 total) | HIGH     |
| H5 | Duplicate Code (general)   | Overall duplicate code %    | <5%  | 5–10%    | >10%      | ~8% (structural + block duplicates)           | MODERATE |

|     |                                        |                                   |      |           |                |                                                          |          |
|-----|----------------------------------------|-----------------------------------|------|-----------|----------------|----------------------------------------------------------|----------|
| H6  | High Churn Areas                       | Monthly changes (top files)       | <5   | 5–10      | >10            | 2 (max per file, 6-month window)                         | GOOD     |
| H7  | Defect-Prone Files                     | Fix commits (hottest file)        | 1–3  | 4–5       | >5             | 1 ( <code>frontend/src/App.tsx</code> , logo fix)        | GOOD     |
| H8  | Ownership Issues                       | Top-author ownership %            | >80% | 60–80%    | <60%           | 100% (single author <code>ksabai-gl</code> on hot files) | GOOD     |
| H9  | Dual API Runtimes (additional)         | Parallel endpoint implementations | 0    | 1 runtime | 2+ full stacks | 2 (Laravel + Express)                                    | HIGH     |
| H10 | Missing Lifecycle Cleanup (additional) | Components with interval/SSE leak | 0    | 1         | 2+             | 1 ( <code>LegacyMonitorPoller.jsx</code> )               | MODERATE |

### Hotspot Score breakdown

| Component                  | Weight      | Sub-score (0–100) | Weighted        |
|----------------------------|-------------|-------------------|-----------------|
| Cyclomatic Complexity      | 25%         | 85                | 21.25           |
| Code Churn                 | 25%         | 10                | 2.50            |
| Defect Density             | 20%         | 15                | 3.00            |
| Class/Function Size        | 15%         | 72                | 10.80           |
| Business Logic Duplication | 10%         | 78                | 7.80            |
| Developer Ownership Risk   | 5%          | 5                 | 0.25            |
| <b>Hotspot Score</b>       | <b>100%</b> |                   | <b>46 / 100</b> |

## 2.5 Actions Required

| Hotspot                       | Action                                                                                                                                              | Rating    | Priority |
|-------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------|-----------|----------|
| H1 High Cyclomatic Complexity | Split <code>ConnectPage.tsx</code> and <code>DiscoveryPage.tsx</code> into sub-components; extract query hooks; target CC <10 per unit              | HIGH RISK | CRITICAL |
| H3 Large Functions            | Decompose 201-line <code>ConnectPage</code> into 4 feature components; move <code>getSeedDocuments</code> fixtures to JSON files                    | HIGH RISK | HIGH     |
| H4 Business Logic Duplication | Consolidate reachability formula, <code>buildTree</code> , and test-runner workflows into shared domain services; deprecate duplicate Express logic | HIGH RISK | CRITICAL |
| H5 Duplicate Code (general)   | Extract <code>TreeBuilder</code> utility; add jscpd/PHPCPD to CI with <5% threshold                                                                 | MODERATE  | MEDIUM   |
| H9 Dual API Runtimes          | Designate single production API runtime; document dev-api deprecation or generate from OpenAPI                                                      | HIGH RISK | HIGH     |

|                               |                                                                                                                                       |          |        |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|----------|--------|
| H10 Missing Lifecycle Cleanup | Add <code>componentWillUnmount</code> to <code>LegacyMonitorPoller.jsx</code> or migrate to hooks with <code>useEffect</code> cleanup | MODERATE | MEDIUM |
|-------------------------------|---------------------------------------------------------------------------------------------------------------------------------------|----------|--------|

## 2.6 Expected Outcomes

---

- Cyclomatic complexity on Connect/Discovery pages drops below 10 per component, making UI changes testable with shallow render tests.
- Business-rule changes (reachability threshold, alert logic) require a single edit in a domain service instead of three coordinated copies.
- Eliminating the dual-runtime pattern halves the API maintenance surface and removes drift between Laravel and Express responses.
- Extracted page sub-components enable Storybook documentation and faster code reviews (<80 LOC per PR file).
- Adding complexity and duplication lint rules to CI prevents regression of hotspots identified in this audit.